

DOCUMENTACION TECNICA

Voz = Telegram: claves por dispositivo y tareas largas

CLAVES-Y-TAREAS-LARGAS.MD

Voz = Telegram: claves por dispositivo y tareas largas

Estado al 2026-10-09. Todo lo de este documento esta **probado en hardware** (parlante ESP32-S3 y Cardputer) salvo donde se dice lo contrario.

Objetivo

Que por voz se le puedan dar a Hermes **las mismas ordenes que por Telegram**, con el historial compartido (la voz escribe en la misma base que Telegram, ver el repo `voice-history`). Para eso hicieron falta tres cosas:

1. **Tareas largas:** si Hermes tarda, el parlante avisa y el resultado llega por Telegram, en vez de cortarse.
2. **Clave por dispositivo:** si la voz puede hacer de todo, el bridge no puede aceptar a cualquiera de la LAN. Cada aparato tiene su propia clave, como la de un WiFi.
3. **Instrucciones nuevas:** se saco la "REGLA DE VOZ" que prohibia tareas por este canal.

1. Claves por dispositivo

Como funciona

- Cada aparato manda `Authorization: Bearer <su clave>`: el parlante al abrir el WebSocket (`Audio.cpp`, `websocketSetup`), el Cardputer en cada `POST /voice`.
- El bridge guarda **solo el SHA-256** de cada clave, fuera de los repos: `~/.config/hermes-bridge/devices.json` (directorio 700, archivo 600). Compara en tiempo constante (`hmac.compare_digest`). Relee el archivo cuando cambia: **no hace falta reiniciar el bridge** al dar de alta o revocar.
- Nunca loguea claves. Loguea quien entro:

```
[auth] ws: dispositivo 'esp32-parlante' (B8:F8:62:D7:59:C4, 192.168.100.155)
[auth] voice: dispositivo 'cardputer' (cardputer, 192.168.100.33)
[auth] voice: RECHAZADO, sin clave valida (prueba, 192.168.100.31)
```

Modos (`BRIDGE_AUTH_MODE` en el plist)

Modo	Sin clave valida	<code>GET :3000/api/generate_auth_token</code>
<code>transition</code> (default del codigo)	se acepta y se loguea un aviso	responde la clave compartida vieja
<code>enforce</code> (vigente desde 2026-10-08 22:42)	401	404

`transition` existe para migrar aparatos sin cortar el servicio. Para volver a el (por ejemplo, si un aparato quedo sin clave y hay que usarlo ya), se cambia la variable en el plist y se reinicia el bridge (`unload` + `load -w`).

Administrar claves (en la Mac mini)

```
cd ~/Proyectos/experimentos/hermes-bridge
python3 devices_admin.py add esp32-parlante # clave larga, para pegar
python3 devices_admin.py add cardputer --tecleable # xxxx-xxxx-xxxx-xxxx-xxxx-xxxx, para tipear
python3 devices_admin.py list
python3 devices_admin.py revoke <nombre> # el bridge lo rechaza desde ya
python3 devices_admin.py enable <nombre>
python3 devices_admin.py delete <nombre>
```

- La clave se muestra **una sola vez**. Anotala al crearla y no la pegues en chats ni logs. Conviene que la cree Carlos, no un agente, para que no pase por ninguna conversacion.
- **Si se pierde, no se recupera** (solo hay hash): `delete` + `add` y cargarla de nuevo en el aparato. Paso el 2026-10-08 con la del Cardputer.
- `--tecleable`: alfabeto de 32 sin caracteres confundibles (sin 0/o, 1/l), 6 grupos de 4 = 120 bits, igual de fuerte que la normal.

Registrados al 2026-10-09: `esp32-parlante` y `cardputer`.

Cargar la clave en el parlante (ESP32)

1. Abrir `http://<IP-del-parlante>/wifi` (hoy 192.168.100.155). El portal corre siempre, tambien con la placa conectada a la WiFi.
2. Recuadro "**Clave del bridge**": clave actual + clave nueva → **Guardar y reiniciar**. La placa se reinicia sola a los 2 s (los encabezados del WebSocket se arman una sola vez al arrancar).
3. Verificar en el log del bridge la linea `dispositivo 'esp32-parlante'`.

Detalles de la ruta (`POST /api/wifi/token` , `WifiManager.cpp`):

- **Solo escribe:** ninguna ruta devuelve la clave.
- Pide la **clave actual** si hay una guardada (403 "La clave actual no coincide"), porque el portal no tiene contraseña y cualquiera en la WiFi podría pisarla.
- La nueva: 16 a 128 caracteres `[A-Za-z0-9_-]` (422 si no).
- El mensaje de error aparece **arriba de todo** de la pagina: es facil no verlo.

⚠ **Trampa (2026-10-08):** la "clave actual" de una placa que nunca recibio una clave por el portal **no es necesariamente** `elato-local-token`. El firmware solo pide clave al bridge si la NVS esta vacia, y esta placa venia de la etapa de la nube de Elato: tenia guardado el **JWT de elatoai.com** (451 caracteres). Se resolvió sacandolo del respaldo de flash completo (`~/elatoai-flash-previo-2026-10-08_210441/` , en la VM) con un script de un solo uso que lo mandaba como `current` sin mostrarlo. Si vuelve a pasar con otra placa: hacer un respaldo con `read_flash` antes de tocarla y buscar ahi el valor de `auth_token`.

Cargar la clave en el Cardputer (Bruce, modulo `hermesvoice`)

1. Bruce → **Hermes Voice** → **Config** → **"Clave: sin clave"**.
2. Tipear la clave `--tecleable` con los guiones. Se ve mientras se escribe (para no errarla); despues nunca se vuelve a mostrar ni se precarga.
3. Hacer una pregunta: el bridge recién lo registra en el primer `POST /voice`.

Se guarda en NVS `hv_conf / token`. Si el bridge responde 401, el Cardputer muestra **"Clave rechazada"**. Detalle en `Bruce/docs/hermes-voice-plan.md`.

2. Tareas largas

Que pasa

1. Hermes usa herramientas → no manda texto. Mientras tanto si manda `: keepalive` cada 10 s y eventos `hermes.tool.progress`, que el bridge ignora.
2. Si pasan **25 s sin texto** (`BRIDGE_LONG_TASK_SILENCE_S`), el parlante dice *"Este proceso es largo, mejor mirá los resultados en Telegram."* y queda libre para otra pregunta. El Cardputer recibe esa misma frase como respuesta.
3. La lectura de Hermes sigue **por fuera de la sesion del aparato** (`_spawn_long_task`), así sobrevive a que el parlante se desconecte (los "No PONG" en reposo cierran la sesion).
4. Al terminar, el resultado va a Telegram con `hermes send --to telegram --file -`: sin LLM, reusa las credenciales del gateway (el bridge no guarda el token del bot) y deja el mensaje en la conversacion de Telegram de Deb, que tiene el contexto si le contestas ahi. Formato:

`Pedido por voz: «...»` + el resultado (recortado a 3900 caracteres; el texto completo queda en el historial).

5. Pregunta y respuesta se guardan en el historial compartido, en la sesión de voz original.

Log de un caso real (2026-10-08):

```
23:55:36 User: ... resumirme la última noticia publicada en el diario infovae.com
23:56:01 Tarea larga: 25 s sin texto de Hermes -> aviso por voz y resultado por Telegram
23:56:15 [tarea larga] termino 14 s despues del aviso - 743 chars, enviando a Telegram
23:56:20 [tarea larga] Telegram: enviado
```

Cancelar (verificado en el código del bridge y de Hermes, 2026-10-08)

- **Antes de los 25 s**, el botón o hablar encima cortan la conexión con Hermes, y Hermes hace *hard interrupt* del agente y de sus procesos (`api_server.py`, `_abandon_agent_task`).
- **Lo que ya hizo no se deshace** (si prendió una luz, queda prendida).
- **Después del aviso**, la tarea sigue a propósito: el botón corta el aviso, no la tarea. Un comando nuevo arranca otra en paralelo. Si se quisiera que el botón también la mate, es un cambio en `_start_long_task`.

Si falla

Hermes informa la causa en el último fragmento del SSE (`finish_reason: "error"` y `"error": {"message": ...}`). El bridge la lee (`ask_hermes_stream(on_error=...)`) y, en una tarea larga, Telegram dice "No se pudo completar: ". `explain_hermes_error` traduce:

Mensaje del proveedor	Lo que dice Telegram
<code>Content Exists Risk</code> (DeepSeek, HTTP 400)	rechazo el contenido por su filtro (temas políticos o sensibles)
<code>Insufficient Balance</code> / HTTP 402	se quedó sin saldo
<code>rate limit</code> / HTTP 429	demasiados pedidos seguidos
otro	el mensaje crudo, recortado

En una **pregunta corta** la causa solo queda en el log (`Hermes cerro con error: ...`): el turno termina igual que antes, para no tocar como cierra el parlante. Caso real que lo motivó: 2026-10-08 23:50, DeepSeek rechazó un resumen de posts de X con "Content Exists Risk" (no era falta de saldo: la API decía `is_available: true`).

3. Las instrucciones de voz

Viven en `BRIDGE_SYSTEM_PROMPT` del plist de la Mac mini (la plantilla del repo, `server/hermes-bridge/com.elato.hermes-bridge.plist`, tiene el mismo texto). Vigente desde 2026-10-08 23:43:

Eres un asistente de voz. Responde siempre en español, sin markdown ni listas: tu respuesta será leída en voz alta por un altavoz. En una charla, responde breve y conversacional (1-3 frases). Por este canal puedes hacer lo mismo que por Telegram: usar herramientas, ejecutar tareas, modificar archivos o controlar la casa. Si una tarea tarda, el sistema avisa solo al usuario que mire Telegram y le manda allí tu respuesta final; no hace falta que lo menciones. Al terminar una tarea, resume el resultado en pocas frases claras.

El default que trae `bridge.py` (sin plist) es solo la primera frase de estilo.

Consecuencia aceptada por Carlos: cualquiera que hable cerca del parlante o use el Cardputer puede dar esas ordenes. La clave protege la red, no el microfono.

Variables nuevas del bridge

Variable	Default	Uso
<code>BRIDGE_AUTH_MODE</code>	<code>transition</code>	<code>enforce</code> rechaza sin clave (vigente)
<code>BRIDGE_DEVICES_FILE</code>	<code>~/.config/hermes-bridge/devices.json</code>	Registro de hashes
<code>BRIDGE_LONG_TASK_SILENCE_S</code>	<code>25</code>	Segundos sin texto para pasar a tarea larga
<code>BRIDGE_LONG_TASK_PHRASE</code>	"Este proceso es largo, ..."	Lo que dice el parlante
<code>BRIDGE_LONG_TASK_TARGET</code>	<code>telegram</code>	Destino de <code>hermes send</code>
<code>HERMES_CLI</code>	<code>~/.hermes/hermes-agent/venv/bin/python</code> <code>-m hermes_cli.main</code>	Para <code>hermes send</code>

Pruebas

```
cd ~/Proyectos/experimentos/hermes-bridge
.venv/bin/python3 tests/test_long_task_and_auth.py # 34/34 al 2026-10-09
```

Usa un Hermes y un `hermes send` simulados y un `state.db` temporal: no toca Telegram ni la base real. Cubre tareas largas (con keepalives, vacias, con error del proveedor), turnos cortos, el historial, las claves y los dos modos.

Volver atras

Que	Como
Aceptar aparatos sin clave	<code>BRIDGE_AUTH_MODE=transition</code> en el plist + reiniciar
Volver a prohibir tareas por voz	restaurar <code>com.elato.hermes-bridge.plist.bak-2026-10-08_234337</code> (instrucciones viejas, ya en <code>enforce</code>). El <code>..._224156</code> es anterior: instrucciones viejas y modo <code>transition</code>
Regrabar el firmware actual del parlante	<code>~/elatoai-firmware-respaldo-2026-10-09/flash.sh</code> (conserva WiFi y clave; copia en la Mac mini, <code>~/Respaldos/elatoai/</code>)
Firmware anterior del parlante	<code>~/elatoai-flash-previo-2026-10-08_210441/flash-completo.bin</code> (4 MB, incluye la WiFi y el JWT viejo : no copiarlo a otros lados)
Firmware anterior del Cardputer	<code>~/cardputer-flash-previo-2026-10-08_213032/flash-completo.bin</code> (8 MB, idem)

Commits: `c707aa9` (bridge), `3b3846c` y `a170fc5` (firmware), `5ef1069` (instrucciones), `e84f700` (causa del error), en `feature/esp32-s3-zero-port`. Cardputer: `1c69df1a` en Bruce (`feature/homeassistant-module`, solo local).